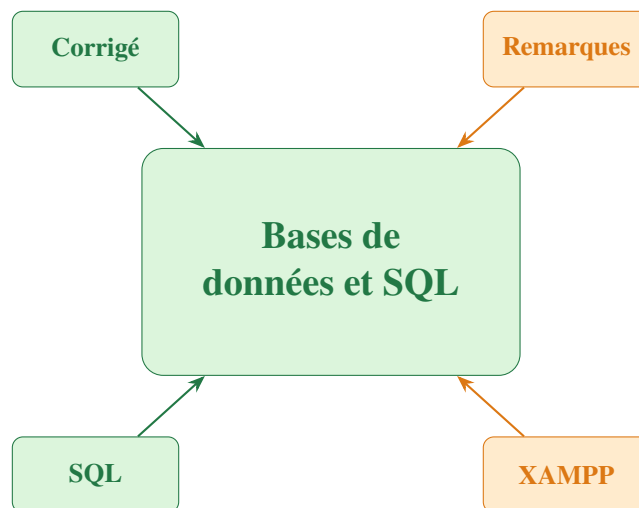


Correction des Travaux Dirigés

Bases de données et SQL

Corrigé détaillé des 26 exercices
avec remarques pédagogiques

Année universitaire 2026 – 2027



Ce document contient les corrections détaillées et les remarques pédagogiques pour les 26 exercices du TD de Bases de données et SQL.

Table des matières

1	Chapitre 1 : Présentation du langage SQL	2
2	Chapitre 2 : Présentation d'une base de données	4
3	Chapitre 3 : Installation de XAMPP	7
4	Chapitre 4 : Les différentes requêtes SQL	9

1 Chapitre 1 : Présentation du langage SQL

Correction — Exercice 1.1 : QCM — Histoire et normes SQL

1. **1. Réponse b)** — SQL a été créé dans les **années 1970**. Les premiers travaux ont été menés par Donald Chamberlin et Raymond Boyce au sein d'IBM à partir de 1974. Le langage, initialement appelé SEQUEL (Structured English Query Language), a été renommé SQL.
2. **2. Réponse b)** — SQL signifie **Structured Query Language** (langage de requêtes structuré). Ce n'est ni un langage standard, ni simple, ni séquentiel.
3. **3. Réponse a)** — La première norme SQL publiée par l'ISO est **SQL-86** (aussi appelée SQL-87 selon les sources). Elle a été suivie de SQL-89, SQL-92, SQL :1999, etc.
4. **4. Réponse b)** — SQL a été initialement développé par **IBM** dans les laboratoires de recherche de San José (Californie). Le prototype s'appelait System R.
5. **5. Réponse b)** — SQL est un langage **déclaratif** : on décrit le résultat souhaité, pas la manière de l'obtenir. Le SGBD se charge de déterminer la stratégie d'exécution optimale.

Remarque

Contrairement aux langages procéduraux (C, Java), SQL est déclaratif : vous spécifiez « quoi » chercher, pas « comment » le chercher. Le SGBD optimise automatiquement l'exécution de la requête.

Correction — Exercice 1.2 : Catégories du langage SQL

Classification :

Instruction	Catégorie
CREATE TABLE	DDL
INSERT INTO	DML
GRANT	DCL
COMMIT	TCL
ALTER TABLE	DDL
UPDATE	DML
REVOKE	DCL
ROLLBACK	TCL
DROP TABLE	DDL
DELETE	DML

Moyen mnémotechnique

- **DDL** (Data Definition Language) : Définir la structure — **CREATE, ALTER, DROP**
- **DML** (Data Manipulation Language) : Manipuler les données — **INSERT, UPDATE, DELETE**
- **DCL** (Data Control Language) : Contrôler les droits — **GRANT, REVOKE**
- **TCL** (Transaction Control Language) : Gérer les Transactions — **COMMIT, ROLLBACK**

Correction — Exercice 1.3 : Vrai ou Faux — Le langage SQL

1. **Faux.** SQL n'est pas un langage propriétaire. C'est un standard ISO/ANSI. Bien qu'Oracle, MySQL, PostgreSQL etc. implémentent SQL avec des variantes, le noyau du langage est normalisé et libre d'utilisation.
2. **Vrai.** SQL couvre deux aspects essentiels : le **DDL** (définir la structure : CREATE, ALTER, DROP) et le **DML** (manipuler les données : INSERT, UPDATE, DELETE, SELECT).
3. **Faux.** SQL n'est pas sensible à la casse pour les mots-clés. SELECT, select et Select sont équivalents. En revanche, les noms de tables/colonnes peuvent être sensibles à la casse selon le SGBD et le système d'exploitation.
4. **Faux.** Le DDL (Data *Definition* Language) concerne la **définition de la structure** des données, pas leur manipulation. La manipulation relève du DML (Data *Manipulation* Language).
5. **Vrai.** Le DCL (Data Control Language) gère les droits d'accès grâce aux instructions GRANT (accorder un droit) et REVOKE (retirer un droit).

Correction — Exercice 1.4 : Tableau récapitulatif**Tableau complété :**

Catégorie	Rôle	Instructions
DDL	Définir et modifier la structure des objets de la base (tables, vues, index)	CREATE, ALTER, DROP, TRUNCATE
DML	Manipuler les données des tables (insérer, modifier, supprimer, interroger)	SELECT, INSERT, UPDATE, DELETE
DCL	Gérer les droits d'accès et les privilèges des utilisateurs	GRANT, REVOKE
TCL	Gérer les transactions (valider, annuler)	COMMIT, ROLLBACK, SAVEPOINT

Remarque

Certains auteurs classent SELECT dans le DQL (Data Query Language), une sous-catégorie du DML dédiée à l'interrogation. Dans la pratique, SELECT est souvent présenté avec le DML.

Correction — Exercice 1.5 : Évolution des normes SQL**1. Principales normes SQL chronologiques :**

- (a) **SQL-86** (1986) — Première norme ANSI/ISO
 - (b) **SQL-89** (1989) — Ajout de l'intégrité référentielle
 - (c) **SQL-92** (1992) — Norme majeure, encore utilisée comme base
 - (d) **SQL :1999** — Ajout des types complexes, récursivité
 - (e) **SQL :2003** — XML, fonctions de fenêtrage
 - (f) **SQL :2006** — Support XML avancé
 - (g) **SQL :2008** — améliorations des types et du trigraphe
 - (h) **SQL :2011** — Temporal tables (tables temporelles)
 - (i) **SQL :2016** — JSON, fonctions polymorphes
 - (j) **SQL :2023** — Dernière norme, améliorations diverses
2. **Nouveauté majeure de SQL :1999** : L'introduction des **types de données complexes** (tableaux, types structurés), de la **récursivité** (CTE récursives) et des **fonctions de fenêtrage** (bien que complètement implémentées dans SQL :2003).
3. **Différence SQL-92 / SQL :1999** : SQL-92 est une norme *relationnelle pure* (tables, lignes, colonnes). SQL :1999 étend le modèle vers l'**orienté objet** (types structurés, héritage) et ajoute la **récursivité** avec les CTE (WITH RECURSIVE).

Remarque

En pratique, aucun SGBD n'implémente intégralement la norme SQL. Chaque SGBD (MySQL, PostgreSQL, Oracle, SQL Server) a ses propres extensions et ses manques. Il est important de consulter la documentation du SGBD utilisé.

2 Chapitre 2 : Présentation d'une base de données**Correction — Exercice 2.1 : QCM — Concepts fondamentaux**

- 1. **1. Réponses b) et d)** — Une base de données est un **ensemble structuré de données interrogeables**, organisée selon le modèle relationnel sous forme de **collection de tables**. Un simple fichier texte ou un tableur ne constituent pas une base de données au sens propre.
- 2. **2. Réponses a) et d)** — Un SGBD sert à **créer, manipuler et administrer** des bases de données, et assure la **sécurité et l'intégrité** des données. Il ne se contente pas de stocker ou lire.
- 3. **3. Réponse b)** — Une ligne d'une table est appelée un **enregistrement** (ou n-uplet, tuple en anglais). Un champ/colonne/attribut désigne une colonne.
- 4. **4. Réponse b)** — Une clé primaire **identifie de façon unique** chaque enregistrement. Elle ne peut pas être NULL et est obligatoirement unique.
- 5. **5. Réponse b)** — Une clé étrangère sert à **établir un lien** entre deux tables en référençant la clé primaire d'une autre table.

Clé primaire vs clé étrangère

La clé primaire est l'identifiant unique d'une table. La clé étrangère est une référence *vers* la clé primaire d'une autre table. Une table peut avoir une seule clé primaire mais plusieurs clés étrangères.

Correction — Exercice 2.2 : Identifier les éléments d'un schéma relationnel**Table LIVRE :**

- Clé primaire : `code_livre`
- Clé étrangère : `code_editeur` (référence EDITEUR)
- Attributs simples : `titre`, `auteur`, `annee_parution`

Table EDITEUR :

- Clé primaire : `code_editeur`
- Clé étrangère : aucune
- Attributs simples : `nom`, `adresse`

Table EMPRUNT :

- Clé primaire : `num_emprunt`
- Clés étrangères : `code_livre` (référence LIVRE), `num_abonne` (référence ABONNE)
- Attributs simples : `date_emprunt`, `date_retour`

Table ABONNE :

- Clé primaire : `num_abonne`
- Clé étrangère : aucune
- Attributs simples : `nom`, `prenom`, `adresse`

Correction — Exercice 2.3 : Types de données SQL**Associations :**

Type SQL	Description
INT	b) Nombre entier
VARCHAR(n)	a) Chaîne de longueur variable, max n caractères
DATE	c) Date (année, mois, jour)
DECIMAL(m, d)	d) Nombre décimal avec m chiffres et d décimales
BOOLEAN	e) Valeur booléenne (vrai/faux)
TEXT	f) Chaîne de longueur illimitée

Remarque

En MySQL, `BOOLEAN` est en réalité un alias pour `TINYINT(1)`. `TEXT` est limité à 65 535 caractères ; pour du texte très long, on utilise `LONGTEXT` (jusqu'à 4 Go).

Correction — Exercice 2.4 : Vrai ou Faux — Bases de données relationnelles

1. **Faux.** La clé primaire est **unique** par définition. Deux enregistrements ne peuvent pas avoir la même valeur de clé primaire — c'est le principe d'unicité.
2. **Vrai.** C'est ce qu'on appelle une **réflexive** (ou auto-référentielle). Par exemple, une table `EMPLOYE` avec une colonne `id_chef` qui référence `id_employe` de la même table.
3. **Vrai.** C'est la définition même de l'**intégrité référentielle** : une clé étrangère doit être soit `NULL` (pas de liaison), soit pointer vers une valeur de clé primaire existante.
4. **Faux.** Le SGBD ne garantit la cohérence que si les contraintes (clés, `NOT NULL`, `CHECK`, etc.) sont définies. Sans contraintes, le SGBD accepte des données incohérentes.
5. **Vrai.** Edgar Frank Codd a publié en 1970 l'article fondateur « A Relational Model of Data for Large Shared Data Banks » dans la revue *Communications of the ACM*.

Correction — Exercice 2.5 : Conception d'un schéma relationnel**Schéma relationnel proposé :**

```
ETUDIANT(num_etudiant, nom, prenom, date_naissance, adresse)
COURS(code_cours, intitule, nombre_heures, #num_enseignant)
ENSEIGNANT(num_enseignant, nom, prenom, specialite)
INSCRIPTION(#num_etudiant, #code_cours)
```

Explications :

- **ETUDIANT** : clé primaire = num_etudiant
- **COURS** : clé primaire = code_cours, clé étrangère = num_enseignant vers **ENSEIGNANT** (un cours est enseigné par un seul enseignant)
- **ENSEIGNANT** : clé primaire = num_enseignant
- **INSCRIPTION** : table de liaison (association several-to-several entre **ETUDIANT** et **COURS**), clé primaire composite = (num_etudiant, code_cours), clés étrangères vers **ETUDIANT** et **COURS**

Remarque

La table **INSCRIPTION** est la table de liaison indispensable pour modéliser l'association several-to-several (n,m) entre étudiants et cours. Sa clé primaire est la concaténation des deux clés étrangères.

Correction — Exercice 2.6 : Avantages d'un SGBD**Principaux avantages d'un SGBD par rapport à un système de fichiers :**

1. **Réduction de la redondance** — Les données sont stockées une seule fois, évitant les duplications.
2. **Intégrité des données** — Contraintes (clés, types, CHECK) garantissant la cohérence.
3. **Partage des données** — Accès simultané par plusieurs utilisateurs.
4. **Sécurité** — Contrôle d'accès, authentification, autorisations (GRANT/REVOKE).
5. **Indépendance données/traitements** — Modification de la structure sans impact sur les applications.
6. **Sauvegarde et récupération** — Mécanismes de backup/restore, gestion des transactions (COMMIT/ROLLBACK).
7. **Langage d'interrogation standardisé** — SQL permet de manipuler les données de manière uniforme.
8. **Concurrence** — Gestion des accès simultanés (verrous, isolation).

Remarque

Les SGBD modernes (MySQL, PostgreSQL, Oracle) offrent en plus des fonctionnalités avancées : triggers, procédures stockées, vues, réplication, partitionnement, etc.

3 Chapitre 3 : Installation de XAMPP

Correction — Exercice 3.1 : QCM — Présentation de XAMPP

1. **Réponses b) et d)** — XAMPP est une **distribution logicielle gratuite** et un **environnement de développement local**. Ce n'est pas un SGBD ni un langage.
2. **Réponse a)** — Les composants principaux de XAMPP sont **Apache, MySQL** (ou MariaDB), **PHP** et **Perl**.
3. **Réponse b)** — Le « X » signifie **Cross-platform** (multiplateforme). XAMPP fonctionne sur Windows, Linux et macOS.
4. **Réponses a) et d)** — Le panneau de contrôle permet de **démarrer/arrêter** les services et de **configurer les ports**. Il ne sert pas à écrire du code ni à créer des bases de données.
5. **Réponse c)** — Par défaut, Apache utilise le port **80** (HTTP). Le port 3306 est celui de MySQL, 8080 est un port alternatif et 3000 est souvent utilisé par Node.js.

Remarque

Retenez les ports par défaut : **Apache = 80** (HTTP), **MySQL = 3306**, **HTTPS = 443**. Si ces ports sont occupés, XAMPP refusera de démarrer le service concerné.

Correction — Exercice 3.2 : Vrai ou Faux — Installation et configuration

1. **Faux**. XAMPP est disponible pour **Windows, Linux** et **macOS**. Le « X » de XAMPP signifie justement cross-platform.
2. **Faux**. Il est **déconseillé** d'installer XAMPP dans **Program Files** car le chemin contient des espaces qui peuvent poser problème. L'emplacement recommandé est **C:\xampp** (Windows).
3. **Vrai**. Le port par défaut de MySQL (MariaDB) dans XAMPP est le **3306**.
4. **Vrai**. Le répertoire **htdocs** est le dossier racine du serveur web Apache. Les fichiers PHP/HTML placés dans ce dossier sont accessibles via l'URL **http://localhost/nom_fichier.php**.
5. **Vrai**. Apache et MySQL sont des services indépendants. On peut démarrer l'un sans l'autre selon les besoins (par exemple, démarrer uniquement MySQL pour utiliser phpMyAdmin).

Correction — Exercice 3.3 : Procédure d'installation de XAMPP

1. **Téléchargement** : Se rendre sur le site officiel **https://www.apachefriends.org** et télécharger la version correspondant à son système d'exploitation.
2. **Choix de version** : Choisir la version contenant la version de PHP souhaitée (par exemple, PHP 8.2). XAMPP propose généralement la dernière version stable.
3. **Lancement de l'installation** : Exécuter le fichier téléchargé (**.exe** sous Windows). Sur Windows, il peut être nécessaire d'exécuter en tant qu'administrateur.
4. **Répertoire recommandé** : **C:\xampp** (Windows) ou **/opt/lampp** (Linux). Éviter les chemins avec espaces.
5. **Vérification** :
 - (a) Ouvrir le panneau de contrôle XAMPP
 - (b) Démarrer Apache
 - (c) Ouvrir un navigateur et aller sur **http://localhost**
 - (d) La page d'accueil XAMPP doit s'afficher
 - (e) Démarrer MySQL et vérifier l'accès à phpMyAdmin via **http://localhost/phpmyadmin**

Remarque

Sur Linux, le panneau de contrôle se lance avec la commande : **sudo /opt/lampp/manager-linux-x64.run**. Les commandes de démarrage/arrêt sont **sudo /opt/lampp/lampp start** et **sudo /opt/lampp/lampp stop**.

Correction — Exercice 3.4 : Accéder à phpMyAdmin

1. **URL d'accès :** Saisir `http://localhost/phpmyadmin` dans le navigateur. Les services Apache et MySQL doivent être démarrés.
2. **Créer une base de données :**
 - (a) Cliquer sur l'onglet **Bases de données**
 - (b) Saisir le nom de la nouvelle base
 - (c) Choisir le classement (`utf8_general_ci` ou `utf8mb4_unicode_ci`)
 - (d) Cliquer sur **Créer**
3. **Créer une table :**
 - (a) Sélectionner la base de données créée
 - (b) Saisir le nom de la table et le nombre de colonnes
 - (c) Définir chaque colonne : nom, type, taille, clé primaire, auto-incrément, etc.
 - (d) Cliquer sur **Sauvegarder**
4. **Avantages de phpMyAdmin :**
 - Interface graphique intuitive (pas besoin de connaître les commandes SQL)
 - Visualisation des tables et données
 - Import/export de bases de données
 - Gestion des utilisateurs et privilèges
 - Exécution de requêtes SQL via l'onglet **SQL**

Remarque

phpMyAdmin est un outil de **gestion** de bases de données MySQL/MariaDB via navigateur. Pour les tâches répétitives ou les scripts, la ligne de commande MySQL reste plus adaptée.

Correction — Exercice 3.5 : Dépannage XAMPP**Causes possibles et solutions :**

1. **Port 80 déjà utilisé** — Un autre programme (Skype, IIS, un autre serveur web) occupe le port 80. *Solution :* Modifier le port d'Apache (par exemple 8080) dans `httpd.conf` ou arrêter le programme conflictuel.
2. **Pare-feu Windows** — Le pare-feu bloque les connexions entrantes. *Solution :* Autoriser Apache dans les règles du pare-feu.
3. **Installation dans un chemin avec espaces** — Le chemin `C:\Program Files\xampp` pose problème. *Solution :* Réinstaller dans `C:\xampp`.
4. **Service déjà en cours d'exécution** — Une instance précédente d'Apache est encore active. *Solution :* Arrêter le service via le gestionnaire de tâches ou la commande `net stop apache2.4`.
5. **Port 443 (HTTPS) bloqué** — Skype ou VMware utilise le port 443. *Solution :* Modifier le port SSL dans `httpd-ssl.conf` ou désactiver SSL.
6. **Permissions insuffisantes** — Sous Linux/macOS, les droits ne permettent pas le lancement. *Solution :* Utiliser `sudo` pour lancer XAMPP.

Remarque

La commande `netstat -ano | findstr :80` (Windows) ou `sudo lsof -i :80` (Linux) permet d'identifier le programme qui occupe le port 80.

4 Chapitre 4 : Les différentes requêtes SQL

Correction — Exercice 4.1 : Requêtes de sélection simples

1. Afficher tous les étudiants :

```
SELECT * FROM ETUDIANT;
```

2. Afficher uniquement les noms et prénoms :

```
SELECT nom, prenom FROM ETUDIANT;
```

3. Afficher les étudiants habitant à Paris :

```
SELECT * FROM ETUDIANT WHERE ville = 'Paris';
```

4. Afficher les étudiants nés après le 1er janvier 2003 :

```
SELECT * FROM ETUDIANT  
WHERE date_naissance > '2003-01-01';
```

5. Afficher les étudiants dont le nom commence par « M » :

```
SELECT * FROM ETUDIANT WHERE nom LIKE 'M%';
```

Remarque

`SELECT *` récupère **toutes** les colonnes. En production, on privilégie `SELECT colonne1, colonne2` pour de meilleures performances. Le symbole `%` dans `LIKE` représente n'importe quelle séquence de caractères.

Correction — Exercice 4.2 : Tri et limitation des résultats

1. Étudiants triés par nom (ordre alphabétique) :

```
SELECT * FROM ETUDIANT ORDER BY nom ASC;
```

2. Étudiants triés par date de naissance décroissante :

```
SELECT * FROM ETUDIANT  
ORDER BY date_naissance DESC;
```

3. Les 3 premiers étudiants par ordre alphabétique :

```
SELECT * FROM ETUDIANT  
ORDER BY nom ASC LIMIT 3;
```

4. Étudiants de Paris triés par prénom :

```
SELECT * FROM ETUDIANT  
WHERE ville = 'Paris'  
ORDER BY prenom ASC;
```

Remarque

`ASC` (ascendant) est la valeur par défaut de `ORDER BY` — on peut l'omettre. `LIMIT` est spécifique à MySQL/MariaDB/PostgreSQL. Sous SQL Server, on utilise `TOP 3`; sous Oracle, `ROWNUM <= 3` ou `FETCH FIRST 3 ROWS ONLY`.

Correction — Exercice 4.3 : Opérateurs logiques et conditions

1. Étudiants habitant Paris ou Lyon :

```
SELECT * FROM ETUDIANT
WHERE ville = 'Paris' OR ville = 'Lyon';
-- Alternative avec IN :
SELECT * FROM ETUDIANT
WHERE ville IN ('Paris', 'Lyon');
```

2. Étudiants nés entre le 01/01/2002 et le 31/12/2003 :

```
SELECT * FROM ETUDIANT
WHERE date_naissance
  BETWEEN '2002-01-01' AND '2003-12-31';
```

3. Étudiants dont le nom contient « ar » :

```
SELECT * FROM ETUDIANT
WHERE nom LIKE '%ar%';
```

4. Étudiants dont la ville commence par une voyelle :

```
SELECT * FROM ETUDIANT
WHERE ville LIKE 'A%'
  OR ville LIKE 'E%'
  OR ville LIKE 'I%'
  OR ville LIKE 'O%'
  OR ville LIKE 'U%'
  OR ville LIKE 'Y%';
-- Alternative avec REGEXP (MySQL) :
SELECT * FROM ETUDIANT
WHERE ville REGEXP '^[AEIOUY]';
```

5. Étudiants ne vivant ni à Paris ni à Lyon :

```
SELECT * FROM ETUDIANT
WHERE ville NOT IN ('Paris', 'Lyon');
-- Alternative :
SELECT * FROM ETUDIANT
WHERE ville <> 'Paris' AND ville <> 'Lyon';
```

Remarque

BETWEEN est inclusif (les bornes sont comprises). **IN** est plus lisible que des **OR** multiples. **NOT IN** exclut les valeurs listées.

Correction — Exercice 4.4 : Fonctions d'agrégat

1. Nombre total d'étudiants :

```
SELECT COUNT(*) AS nb_etudiants  
FROM ETUDIANT;
```

2. Nombre d'étudiants habitant à Paris :

```
SELECT COUNT(*) AS nb_paris  
FROM ETUDIANT WHERE ville = 'Paris';
```

3. Moyenne des notes :

```
SELECT AVG(note) AS moyenne_notes  
FROM INSCRIPTION;
```

4. Note maximale :

```
SELECT MAX(note) AS note_max  
FROM INSCRIPTION;
```

5. Note minimale :

```
SELECT MIN(note) AS note_min  
FROM INSCRIPTION;
```

Remarque

Les fonctions d'agrégat opèrent sur un **ensemble de lignes** et retournent une seule valeur. **COUNT (*)** compte toutes les lignes, **COUNT(colonne)** ignore les NULL. **AVG** ignore également les NULL — attention si toutes les notes ne sont pas renseignées.

Correction — Exercice 4.5 : Regroupement

1. Nombre d'étudiants par ville :

```
SELECT ville, COUNT(*) AS nb_etudiants
FROM ETUDIANT
GROUP BY ville;
```

2. Moyenne des notes par cours :

```
SELECT code_cours, AVG(note) AS moyenne
FROM INSCRIPTION
GROUP BY code_cours;
```

3. Cours ayant une moyenne supérieure à 12 :

```
SELECT code_cours, AVG(note) AS moyenne
FROM INSCRIPTION
GROUP BY code_cours
HAVING AVG(note) > 12;
```

4. Villes ayant plus de 1 étudiant :

```
SELECT ville, COUNT(*) AS nb_etudiants
FROM ETUDIANT
GROUP BY ville
HAVING COUNT(*) > 1;
```

5. Nombre d'inscriptions par étudiant :

```
SELECT num_etudiant,
       COUNT(*) AS nb_inscriptions
FROM INSCRIPTION
GROUP BY num_etudiant;
```

WHERE vs HAVING

WHERE filtre les lignes **avant** le regroupement. **HAVING** filtre les groupes **après** le regroupement. On utilise **HAVING** avec les fonctions d'agrégat, jamais **WHERE**. Ordre d'exécution : **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **ORDER BY**.

Correction — Exercice 4.6 : Jointures

1. Noms des étudiants et intitulé des cours auxquels ils sont inscrits :

```
SELECT e.nom, e.prenom, c.intitule
FROM ETUDIANT e
INNER JOIN INSCRIPTION i
  ON e.num_etudiant = i.num_etudiant
INNER JOIN COURS c
  ON i.code_cours = c.code_cours;
```

2. Noms des enseignants et cours qu'ils dispensent :

```
SELECT ens.nom, ens.prenom, c.intitule
FROM ENSEIGNANT ens
INNER JOIN COURS c
  ON ens.num_enseignant = c.num_enseignant;
```

3. Noms des étudiants et leurs notes (y compris sans note) :

```
SELECT e.nom, e.prenom, i.note
FROM ETUDIANT e
LEFT JOIN INSCRIPTION i
  ON e.num_etudiant = i.num_etudiant;
```

4. Nom de l'étudiant, intitulé du cours et nom de l'enseignant :

```
SELECT e.nom, c.intitule, ens.nom AS enseignant
FROM INSCRIPTION i
INNER JOIN ETUDIANT e
  ON i.num_etudiant = e.num_etudiant
INNER JOIN COURS c
  ON i.code_cours = c.code_cours
INNER JOIN ENSEIGNANT ens
  ON c.num_enseignant = ens.num_enseignant;
```

INNER JOIN vs LEFT JOIN

INNER JOIN retourne uniquement les lignes qui ont une correspondance dans les deux tables. **LEFT JOIN** retourne **toutes** les lignes de la table de gauche, même sans correspondance à droite (les colonnes de droite seront NULL). Pour voir les étudiants **non inscrits**, **LEFT JOIN** est indispensable.

Correction — Exercice 4.7 : Sous-requêtes

1. Étudiants ayant une note supérieure à la moyenne générale :

```
SELECT e.nom, e.prenom, i.note
FROM ETUDIANT e
INNER JOIN INSCRIPTION i
  ON e.num_etudiant = i.num_etudiant
WHERE i.note > (
  SELECT AVG(note) FROM INSCRIPTION
);
```

2. Cours dont le nombre d'heures est supérieur à la moyenne :

```
SELECT code_cours, intitule, nombre_heures
FROM COURS
WHERE nombre_heures > (
  SELECT AVG(nombre_heures) FROM COURS
);
```

3. Étudiants inscrits au cours « Base de données » :

```
SELECT e.nom, e.prenom
FROM ETUDIANT e
WHERE e.num_etudiant IN (
  SELECT i.num_etudiant
  FROM INSCRIPTION i
  INNER JOIN COURS c
    ON i.code_cours = c.code_cours
  WHERE c.intitule = 'Base de donnees'
);
```

4. Enseignant qui dispense le cours avec la meilleure moyenne :

```
SELECT ens.nom, ens.prenom
FROM ENSEIGNANT ens
WHERE ens.num_enseignant = (
  SELECT c.num_enseignant
  FROM COURS c
  INNER JOIN INSCRIPTION i
    ON c.code_cours = i.code_cours
  GROUP BY c.code_cours, c.num_enseignant
  ORDER BY AVG(i.note) DESC
  LIMIT 1
);
```

Remarque

Une sous-requête est une requête **imbriquée** à l'intérieur d'une autre. On distingue : (1) les sous-requêtes scalaires (retournant une seule valeur, utilisées avec >, =, etc.) et (2) les sous-requêtes ensemblistes (retournant plusieurs valeurs, utilisées avec IN, NOT IN). On peut souvent remplacer une sous-requête par une jointure.

Correction — Exercice 4.8 : Insertion, mise à jour et suppression

1. Insérer un nouvel étudiant :

```
INSERT INTO ETUDIANT
(num_etudiant, nom, prenom,
 date_naissance, ville)
VALUES
(6, 'Durand', 'Alice',
 '2004-01-15', 'Lille');
```

2. Inscrire l'étudiant 6 au cours « BD » :

```
INSERT INTO INSCRIPTION
(num_etudiant, code_cours, note)
VALUES (6, 'BD', NULL);
```

3. Modifier la ville de l'étudiant 3 :

```
UPDATE ETUDIANT
SET ville = 'Nice'
WHERE num_etudiant = 3;
```

4. Mettre à jour la note de l'étudiant 1 au cours « BD » :

```
UPDATE INSCRIPTION
SET note = 15
WHERE num_etudiant = 1
AND code_cours = 'BD';
```

5. Supprimer l'inscription de l'étudiant 5 :

```
DELETE FROM INSCRIPTION
WHERE num_etudiant = 5;
```

6. Supprimer tous les étudiants de Toulouse :

```
DELETE FROM ETUDIANT
WHERE ville = 'Toulouse';
```

Précautions avec DELETE et update

Toujours utiliser WHERE avec UPDATE et DELETE. Sans WHERE, **toutes les lignes** de la table seront modifiées ou supprimées ! Avant un DELETE, faire un SELECT avec le même WHERE pour vérifier les lignes affectées.

Correction — Exercice 4.9 : Création et modification de tables1. Créer la table **ETUDIANT** :

```
CREATE TABLE ETUDIANT (  
    num_etudiant INT PRIMARY KEY,  
    nom VARCHAR(50) NOT NULL,  
    prenom VARCHAR(50) NOT NULL,  
    date_naissance DATE,  
    ville VARCHAR(50)  
);
```

2. Créer la table **INSCRIPTION** avec clés étrangères :

```
CREATE TABLE INSCRIPTION (  
    num_etudiant INT,  
    code_cours VARCHAR(10),  
    note DECIMAL(4,2),  
    PRIMARY KEY (num_etudiant, code_cours),  
    FOREIGN KEY (num_etudiant)  
        REFERENCES ETUDIANT(num_etudiant),  
    FOREIGN KEY (code_cours)  
        REFERENCES COURS(code_cours)  
);
```

3. Ajouter la colonne **email** à **ETUDIANT** :

```
ALTER TABLE ETUDIANT  
ADD email VARCHAR(100);
```

4. Modifier **nombre_heures** pour qu'il ne puisse pas être **NULL** :

```
ALTER TABLE COURS  
MODIFY nombre_heures INT NOT NULL;
```

5. Supprimer la colonne **specialite** de **ENSEIGNANT** :

```
ALTER TABLE ENSEIGNANT  
DROP COLUMN specialite;
```

Remarque

PRIMARY KEY implique **NOT NULL** et **UNIQUE**. **FOREIGN KEY** assure l'intégrité référentielle : on ne peut pas insérer une inscription avec un **num_etudiant** qui n'existe pas dans **ETUDIANT**. **MODIFY** est spécifique à MySQL ; sous PostgreSQL, on utilise **ALTER COLUMN ... SET NOT NULL**.

Correction — Exercice 4.10 : Requête combinée**Requête :**

```
SELECT e.ville,  
       COUNT(*) AS nb_etudiants,  
       AVG(i.note) AS moyenne_notes  
FROM ETUDIANT e  
INNER JOIN INSCRIPTION i  
  ON e.num_etudiant = i.num_etudiant  
GROUP BY e.ville  
HAVING COUNT(*) >= 2  
ORDER BY moyenne_notes DESC;
```

Explication étape par étape :

1. **INNER JOIN** : on relie **ETUDIANT** et **INSCRIPTION** pour accéder aux notes
2. **GROUP BY ville** : on regroupe les résultats par ville
3. **COUNT(*)** : on compte le nombre d'étudiants par ville
4. **AVG(note)** : on calcule la moyenne des notes par ville
5. **HAVING COUNT(*) >= 2** : on ne conserve que les villes ayant au moins 2 étudiants
6. **ORDER BY moyenne_notes DESC** : on trie par moyenne décroissante

Remarque

Cette requête combine plusieurs notions : jointure, agrégat, regroupement, filtrage de groupes et tri. C'est le type de requête qu'on rencontre souvent en entreprise. La clause **HAVING** filtre sur le résultat de l'agrégat, tandis que **WHERE** filtre sur les lignes individuelles.